

AWS NOTES

INTERVIEW & CONCEPT GUIDE

Core AWS services explained | 46 questions across 9 areas

Cloud Computing & AWS Fundamentals	5 questions
IAM & Security	6 questions
Compute: EC2, Auto Scaling & Containers	5 questions
Storage: S3, EBS & EFS	5 questions
Databases: RDS, DynamoDB & Aurora	5 questions
Serverless: Lambda & Event-Driven Compute	5 questions
Networking: VPC, CloudFront & Route 53	5 questions
Messaging & Event-Driven Architecture: SQS, SNS & EventBridge	5 questions
Monitoring & Operations: CloudWatch, CloudTrail & Cost	5 questions

Prepared for: CJDC - Creative Java Developers Community
Slogan: Grow Together

Contents

1. Cloud Computing & AWS Fundamentals	5 questions
2. IAM & Security	6 questions
3. Compute: EC2, Auto Scaling & Containers	5 questions
4. Storage: S3, EBS & EFS	5 questions
5. Databases: RDS, DynamoDB & Aurora	5 questions
6. Serverless: Lambda & Event-Driven Compute	5 questions
7. Networking: VPC, CloudFront & Route 53	5 questions
8. Messaging & Event-Driven Architecture: SQS, SNS & EventBridge	5 questions
9. Monitoring & Operations: CloudWatch, CloudTrail & Cost	5 questions

How to use this guide

- These are conceptual/explanation questions - practice saying the answer out loud in under a minute.
- For any 'when would you choose X over Y' question, always state the trade-off, not just the definition.
- Draw the architecture whenever you can - AWS interviewers consistently reward diagrams over pure verbal answers.
- Revisit the sections you're weakest in a second time rather than only reading once top to bottom.

Q1. What is AWS and what problem does it solve?

AWS is Amazon's cloud platform: a set of on-demand compute, storage, networking, and managed services billed by usage. It removes the need to buy, rack, and maintain physical servers, letting teams provision infrastructure in minutes and pay only for what they consume.

Q2. What are Regions, Availability Zones, and Edge Locations?

A Region is a geographic area (e.g., ap-south-1) containing multiple isolated data centers called Availability Zones (AZs), each with independent power and networking so a failure in one AZ doesn't take down the others. Edge Locations are smaller, globally distributed points used by CloudFront to cache content closer to end users.

Q3. Explain IaaS, PaaS, and SaaS with AWS examples.

IaaS gives you raw virtualized infrastructure you manage yourself (EC2, EBS). PaaS gives you a managed runtime where AWS handles the underlying servers (Elastic Beanstalk, RDS). SaaS is a fully finished application you just use (e.g., Amazon Chime). The trade-off going IaaS to SaaS is less operational control for less operational burden.

Q4. What is the AWS Shared Responsibility Model?

AWS secures 'of the cloud' - physical data centers, hardware, network infrastructure, and the virtualization layer. The customer secures 'in the cloud' - OS patching, IAM policies, data encryption, security group rules, and application-level security. For managed services like RDS or Lambda, AWS takes on more of that burden than it does for raw EC2.

Q5. How would you design a highly available web application on AWS at a high level?

Route 53 for DNS with health-check-based failover, CloudFront in front for caching static assets, an Application Load Balancer distributing traffic across EC2 instances (or containers) spread across multiple AZs inside an Auto Scaling Group, and a Multi-AZ RDS instance or DynamoDB for the data layer so no single AZ failure takes the app down.

Q1. What is IAM and what are its core components?

IAM (Identity and Access Management) controls who can do what in an AWS account. Core components: Users (individual identities), Groups (collections of users sharing permissions), Roles (identities assumed temporarily, commonly by services or federated users), and Policies (JSON documents that define allowed/denied actions on specific resources).

Q2. What's the difference between an IAM User and an IAM Role?

A User has long-lived credentials tied to a specific person or application. A Role has no long-term credentials at all - it's assumed temporarily and grants short-lived, auto-rotating credentials. Roles are the recommended way to grant permissions to EC2 instances, Lambda functions, or federated users, since there's no static secret to leak or rotate.

Q3. What is the principle of least privilege and how do you enforce it in IAM?

Grant only the exact permissions an identity needs to do its job, nothing more. In practice: write scoped policies that list specific actions and resource ARNs instead of using '*', attach policies to groups/roles rather than individual users, and periodically review access with IAM Access Analyzer to catch unused or overly broad permissions.

Q4. What is MFA, and why should the root account always have it enabled?

Multi-Factor Authentication requires a second proof of identity (an authenticator app code or hardware key) beyond a password. The root account has unrestricted access to the entire AWS account including billing, so if its password alone is compromised, the whole account is compromised - MFA on root is considered a non-negotiable baseline.

Q5. How do you manage secrets like database passwords and API keys securely on AWS?

Never hardcode them in code or environment files committed to source control. Use AWS Secrets Manager (built-in rotation, fine-grained IAM access) or Systems Manager Parameter Store (cheaper, good for less sensitive config), and grant access via IAM roles rather than distributing static keys to each service.

Q6. What does CloudTrail do, and how is it different from CloudWatch?

CloudTrail is an audit log: it records every API call made in the account - who did what, from where, and when - which is essential for security investigations and compliance. CloudWatch is about operational monitoring: metrics, logs from running applications, and alarms. CloudTrail answers 'who changed this,' CloudWatch answers 'how is this behaving.'

Q1. What is EC2 and what are the main instance family categories?

EC2 (Elastic Compute Cloud) provides resizable virtual servers. Main families: General Purpose (balanced, e.g. t3/m5) for typical web apps, Compute Optimized (c-series) for CPU-heavy workloads, Memory Optimized (r-series) for in-memory databases and caches, Storage Optimized (i/d-series) for high-IOPS workloads, and Burstable (t-series) for variable, spiky traffic where sustained full CPU isn't needed.

Q2. What is the difference between stopping and terminating an EC2 instance?

Stopping shuts the instance down but preserves its EBS root volume and instance ID, so you can start it again later with the same data (though you lose any instance-store/ephemeral data and get billed for the EBS storage while stopped). Terminating permanently deletes the instance and, by default, its root EBS volume unless you've set it not to delete on termination.

Q3. How does EC2 Auto Scaling decide when to add or remove instances?

You define a Launch Template (AMI, instance type, security groups) and a Scaling Policy tied to a CloudWatch metric such as average CPU utilization or requests-per-target. When the metric crosses your threshold for the configured evaluation period, Auto Scaling adds (scales out) or removes (scales in) instances within your configured min/max bounds, and replaces any instance that fails its health check.

Q4. What is the difference between an Application Load Balancer and a Network Load Balancer?

ALB operates at Layer 7 (HTTP/HTTPS) and can route based on path, host header, or query string - ideal for web applications and microservices. NLB operates at Layer 4 (TCP/UDP), handles extreme throughput and preserves the client's source IP, and is used for latency-sensitive or non-HTTP protocols.

Q5. When would you choose ECS/EKS over plain EC2 for running an application?

Once you're deploying many independently-scaled services and need consistent packaging, container orchestration (ECS with Fargate, or EKS/Kubernetes) gives you automated scheduling, self-healing, rolling deployments, and better resource utilization than manually managing EC2 fleets per service. Fargate specifically removes the need to manage the underlying EC2 hosts at all - you just define the container spec.

Q1. What is S3 and what problem is it designed to solve?

S3 (Simple Storage Service) is object storage designed for effectively unlimited, highly durable (11 nines) storage of arbitrary files, accessed over HTTP(S) rather than mounted as a filesystem. It's the default choice for static assets, backups, data lake storage, and hosting static websites.

Q2. What is the difference between S3 Standard, S3 Standard-IA, and S3 Glacier?

S3 Standard is for frequently accessed data at the highest per-GB cost. Standard-IA (Infrequent Access) costs less per GB but adds a retrieval fee, suited to backups accessed occasionally. Glacier tiers (Instant, Flexible, Deep Archive) are the cheapest, meant for long-term archival where retrieval can take minutes to hours - you trade retrieval speed for storage cost.

Q3. What is S3 Versioning and why would you enable it?

Versioning keeps every version of an object instead of overwriting it, so an accidental delete or overwrite can be recovered by restoring a prior version. It's commonly paired with a Lifecycle policy that automatically moves or expires older versions to control storage cost growth.

Q4. What's the difference between EBS and instance store volumes?

EBS is network-attached block storage that persists independently of the EC2 instance's lifecycle and can be detached/reattached or snapshotted. Instance store is physically attached to the host and is high-performance but ephemeral - its data is lost if the instance stops or the underlying hardware fails, so it's only suited for temporary/cache data.

Q5. When would you use EFS instead of EBS?

EBS is a block volume attachable to exactly one instance at a time (per AZ). EFS is a managed NFS file system that multiple EC2 instances across AZs can mount concurrently - use it when several servers need shared, simultaneous read/write access to the same files, such as a shared content directory behind a fleet of web servers.

Q1. What is RDS and which engines does it support?

RDS (Relational Database Service) is a managed relational database that handles provisioning, patching, backups, and failover for you. Supported engines include MySQL, PostgreSQL, MariaDB, Oracle, SQL Server, and Aurora (AWS's own MySQL/PostgreSQL-compatible engine).

Q2. What is Multi-AZ deployment in RDS, and how is it different from a Read Replica?

Multi-AZ maintains a synchronous standby copy in a different AZ purely for failover - the standby isn't used for reads and automatic failover happens if the primary becomes unhealthy. Read Replicas are asynchronous copies you can actually query, used to horizontally scale read traffic away from the primary; they can even be promoted to standalone databases but aren't meant for automatic failover.

Q3. When would you choose DynamoDB over RDS?

Choose DynamoDB when you need single-digit-millisecond latency at very large, unpredictable scale, your access patterns are known in advance and key-based (not complex ad-hoc joins), and you want a serverless, auto-scaling database with no capacity planning. Choose RDS when you need complex relational queries, joins, and strict ACID transactions across many related tables.

Q4. What is a DynamoDB partition key, and why does its design matter?

The partition key determines which physical partition an item is stored on; DynamoDB hashes it to spread data (and therefore read/write load) across partitions. A poorly chosen key with low cardinality (e.g., a status field with only 3 possible values) creates 'hot partitions' that throttle throughput, so keys should be chosen for even access distribution.

Q5. What makes Aurora different from standard RDS MySQL/PostgreSQL?

Aurora separates compute from a distributed, self-healing storage layer replicated six ways across three AZs, giving it faster failover, higher throughput, and better crash recovery than standard RDS - AWS advertises it as several times faster than stock MySQL at a comparable cost, largely because the storage engine itself is re-architected, not just a managed EC2 instance running MySQL.

Q1. What is AWS Lambda and how does its pricing model work?

Lambda runs your code in response to events without you provisioning or managing any server. You're billed per invocation and for the compute time actually consumed (measured in milliseconds), not for idle capacity - which makes it economical for spiky or infrequent workloads compared to a server that runs 24/7.

Q2. What is a 'cold start' in Lambda and how do you reduce its impact?

A cold start is the extra latency incurred when Lambda has to initialize a new execution environment (download code, start the runtime) before running your function, as opposed to reusing a warm one. You can reduce it by keeping deployment packages small, avoiding heavy initialization work outside the handler, choosing a faster-starting runtime, or using Provisioned Concurrency to keep a pool of environments pre-warmed.

Q3. What are common event sources that trigger a Lambda function?

S3 (object created/deleted), API Gateway (HTTP requests), DynamoDB Streams (table changes), SQS/SNS (queued or broadcast messages), EventBridge (scheduled or custom events), and CloudWatch Events/Logs, among others - Lambda is designed to glue together many AWS services reactively.

Q4. How do you handle errors and retries for an asynchronously invoked Lambda function?

Asynchronous invocations (like from S3 or SNS) are automatically retried by Lambda on failure; after retries are exhausted, you should configure a Dead Letter Queue or a Lambda Destination (on-failure) so the failed event isn't silently dropped and can be inspected or reprocessed.

Q5. What does 'serverless' actually mean - are there really no servers involved?

There are still servers - AWS just owns and manages them entirely, so your team never provisions, patches, or scales the underlying machines. 'Serverless' describes the absence of operational server management from the developer's perspective, not the literal absence of hardware.

Q1. What is a VPC and what are its core building blocks?

A VPC (Virtual Private Cloud) is your own logically isolated network inside AWS, defined by a CIDR block. Core building blocks: Subnets (public or private slices of the VPC's IP range), Route Tables (control where traffic goes), an Internet Gateway (lets public subnets reach the internet), a NAT Gateway (lets private subnets initiate outbound internet traffic without being reachable from it), Security Groups (stateful instance-level firewall), and Network ACLs (stateless subnet-level firewall).

Q2. What is the difference between a Security Group and a Network ACL?

A Security Group is stateful and attached to individual instances/ENIs - if you allow inbound traffic, the matching outbound response is automatically allowed. A Network ACL is stateless and attached at the subnet level - you must explicitly allow both inbound and outbound rules, and rules are evaluated in numbered order, making NACLs useful as a coarse, subnet-wide additional layer of defense.

Q3. Why would a private subnet need a NAT Gateway?

Instances in a private subnet have no public IP and aren't directly reachable from the internet, which is intentional for backend/database tiers. But they may still need outbound access - for OS updates or calling an external API - so a NAT Gateway in a public subnet lets them initiate outbound connections while remaining unreachable from the outside.

Q4. What does CloudFront do, and how does it work with an S3 origin?

CloudFront is AWS's CDN: it caches content at edge locations around the world so users get it from a nearby location instead of the origin server, reducing latency and origin load. With an S3 origin, you'd typically use Origin Access Control so the bucket is only reachable through CloudFront, not directly, letting you enforce HTTPS, signed URLs, and caching rules at the edge.

Q5. What are Route 53's main routing policies, and when would you use Weighted vs Failover routing?

Simple (one resource), Weighted (split traffic by percentage - useful for canary releases), Latency-based (route to the region with lowest latency for the user), Failover (route to a backup resource if health checks on the primary fail), Geolocation, and Geoproximity. Use Weighted for gradually shifting traffic to a new version; use Failover specifically for active-passive disaster recovery setups.

Messaging & Event-Driven Architecture: SQS, SNS & EventBridge

Section 8 of 9

Q1. What is the difference between SQS and SNS?

SQS is a point-to-point queue: a message is stored until exactly one consumer processes and deletes it, making it ideal for decoupling a producer from a worker pool doing background jobs. SNS is publish/subscribe: one message published to a topic can fan out to many subscribers (SQS queues, Lambda, email, HTTP endpoints) simultaneously, making it ideal for broadcasting an event to multiple interested consumers.

Q2. What is a Dead Letter Queue and why is it important?

When a consumer repeatedly fails to process a message from a queue, instead of retrying forever (blocking the queue) or silently losing it, the message is moved to a DLQ after a configured retry count. This lets you alert on and inspect failed messages without one bad message stalling processing of everything behind it.

Q3. What is the difference between SQS Standard and FIFO queues?

Standard queues offer at-least-once delivery with best-effort ordering and virtually unlimited throughput - duplicate or out-of-order delivery is possible. FIFO queues guarantee exactly-once processing and strict ordering within a message group, at the cost of lower throughput - use FIFO when the order of operations actually matters (e.g., sequential order updates for the same customer).

Q4. What is EventBridge and how is it different from SNS?

EventBridge is an event bus built for routing structured events between AWS services, SaaS partners, and your own applications, using content-based filtering rules to decide which targets receive which events. SNS is simpler pub/sub with less sophisticated routing logic; EventBridge is generally preferred for building larger event-driven architectures with many producers and rule-based routing.

Q5. How would you design a resilient order-processing pipeline using SQS and Lambda?

An order event is published to an SNS topic, fanned out to an SQS queue that buffers work for a Lambda consumer. The consumer processes messages idempotently, deletes them from the queue on success, and a DLQ captures anything that fails repeatedly - this way a burst of orders is smoothed out by the queue rather than overwhelming downstream services directly.

Q1. What does CloudWatch monitor, and what are its main components?

CloudWatch collects Metrics (numeric time-series data like CPU utilization), Logs (centralized application/service logs), Alarms (trigger notifications or actions when a metric crosses a threshold), and Dashboards (visualize metrics together). It's the primary operational monitoring tool across almost every AWS service.

Q2. How would you set up alerting so your team knows about a production issue before customers report it?

Define CloudWatch Alarms on key metrics (error rate, latency p99, queue depth, CPU) with sensible thresholds, route alarm state changes through SNS to a paging tool like PagerDuty or a Slack channel, and layer in synthetic canaries or health checks that proactively test critical user flows rather than only relying on infrastructure-level metrics.

Q3. What is the AWS Well-Architected Framework, and what are its pillars?

It's a set of best-practice guidelines for evaluating cloud architectures across six pillars: Operational Excellence, Security, Reliability, Performance Efficiency, Cost Optimization, and Sustainability. Interviewers often ask you to reason about a design decision through the lens of one or more of these pillars rather than just 'does it work.'

Q4. What are some concrete ways to reduce AWS costs without hurting reliability?

Right-size instances based on actual utilization rather than guesswork, use Auto Scaling so you're not paying for idle peak-capacity 24/7, move infrequently accessed S3 data to cheaper storage tiers via lifecycle policies, use Reserved Instances or Savings Plans for predictable steady-state workloads, and use Spot Instances for fault-tolerant, interruptible batch work.

Q5. How do you troubleshoot a sudden latency spike in a production AWS application?

Start with CloudWatch metrics for the relevant tier (ALB target response time, EC2 CPU/network, RDS connections and CPU) to localize which layer is slow, check CloudWatch Logs and any distributed tracing (X-Ray) around the same timestamps to see where time is being spent within a request, and check CloudTrail/recent deployments for a correlated change that might have introduced the regression.